MultiSky: Dynamic Resource Allocation Framework for High-Throughput CGRA Multitask Execution

Yufei Yang, Chenhao Xie, Rui Wang, *Senior Member, IEEE*, Liansheng Liu, *Member, IEEE*, Xiyuan Peng, and Yu Peng

***Abstract*—Coarse-grained reconfigurable arrays (CGRAs) offer a promising balance between high performance and flexibility, yet dynamic resource allocation in multitask scenarios remains challenging due to unpredictable task creation/destruction. Existing static approaches lack flexibility, while dynamic methods suffer from high latency or limited applicability. This article presents MultiSky, a framework for CGRA multitask dynamic resource allocation, combining a hardware controller and a software premapper. The hardware controller dynamically allocates resources within hundreds of cycles by calculating tile allocation for each task via weighted averaging, and generating tile shapes using a lightweight heuristic algorithm. The software premapper employs incremental compilation to pregenerate configurations, avoiding online transformation overhead. Evaluations on a real-world multitask scenario demonstrate that MultiSky achieves 1.72× higher throughput than baselines by maintaining 82.7% average resource utilization. The framework scales efficiently with larger CGRAs and task counts, with hardware overhead decreasing to 1% for 16 × 16 CGRAs. These results highlight MultiSky's ability to balance flexibility, efficiency, and practicality in dynamic computing environments.**

***Index Terms*—Coarse-grained reconfigurable arrays (CGRAs), CGRA mapper, dynamic resource allocation, multitask.**

I. Introduction

IN MODERN integrated circuit design, the growing demands for computing performance and energy efficiency are highlighting the limitations of traditional fixed-architecture processors. Coarse-grained reconfigurable arrays (CGRAs) have emerged to meet these need [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12]. CGRA combines the high performance of application-specific integrated circuits (ASICs) with the flexibility of field programmable gate arrays (FPGAs). Compared with ASICs, which involve complex processes from design to tape-out, CGRA stands out with its simplicity. It can be easily configured for various computing applications without having to repeat such elaborate procedures. Compared with FPGAs, CGRA has higher computing performance and energy efficiency [13]. These features give CGRA great application potential in areas like AI acceleration [14], digital signal processing [15], and edge computing [16], [17].

To fully unlock the application potential of CGRA in these areas, understanding the nature of concurrent computational tasks becomes imperative. In modern-day computing scenarios, the execution of multiple tasks simultaneously is not just a convenience but a necessity [18], [19], [20], [21]. The resource requirements for these tasks are constantly changing, as unpredictable external environmental changes can lead to random construction/destruction of tasks. For example, in autonomous driving, object detection and path-planning tasks run concurrently. New objects or traffic situations create new tasks, and user actions like changing destinations also affect tasks. In streaming, video and audio processing happen together, and user-requested video quality changes impact resources. In IoT monitoring, sensor-related tasks vary randomly due to malfunctions or user-adjusted parameters. Thus, dynamic resource allocation in CGRA is key for higher resource utilization, which may further improve the multitask throughput.

However, the existing research on generating resource allocation decisions for dynamic resource allocation falls short. Static approaches [22], [23], [24], [25], [26], [27] are overly reliant on offline profiling of each individual task. This involves painstakingly analyzing the possible dynamic resource demands and how different tasks might combine. For instance, in a system with numerous built-in tasks, like a comprehensive IoT edge device with hundreds of sensor-related tasks, this offline analysis becomes extremely labor-intensive. It's simply not practical for modern, fast-paced computing scenarios. As for dynamic approaches [28], [29], [30], some are too simplistic to manage the intense resource contention and release that occur during the random construction/destruction of multiple tasks. Others can only be applied to tasks where input characteristic variations are crucial, for example, sparse matrix multiplication that is sensitive to the number of zeros in the streaming input, which limits their scope of application.

To tackle this challenge, we present MultiSky, a framework designed for dynamic resource allocation in CGRA multitask scenarios. As depicted in Fig. 1, at the hardware layer, a MultiSky controller plays a pivotal role each time the dynamic resource allocation is triggered by the CPU. First, it calculates the number of resources (CGRA tiles) to be allocated to each task using a weighted-average method. Subsequently, a

Received 27 February 2025; revised 7 June 2025 and 20 July 2025; accepted 5 August 2025. Date of publication 11 August 2025; date of current version 23 February 2026. This work was supported in part by the Provincial Natural Science Foundation of China under Grant LH2023F103. This article was recommended by Associate Editor W. Zhang. *(Corresponding author: Yu Peng.)*

Yufei Yang, Liansheng Liu, Xiyuan Peng, and Yu Peng are with the School of Electronics and Information Engineering, Harbin Institute of Technology, Harbin 150001, Heilongjiang, China (e-mail: pengyu@hit.edu.cn).

Chenhao Xie and Rui Wang are with the School of Computer Science and Engineering, Beihang University, Beijing 100191, China.

Digital Object Identifier 10.1109/TCAD.2025.3597525

1937-4151 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

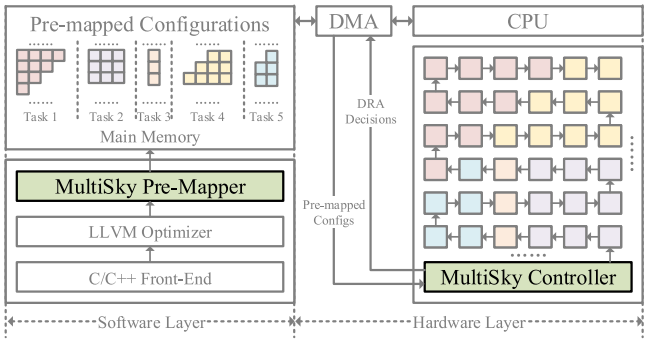

Fig. 1. MultiSky is established based on the existing CGRA system. A MultiSky controller and a dedicated bus are integrated into the hardware layer and the software layer is equipped with an additional MultiSky premapper (labeled in green).

lightweight algorithm is designed to generate the shape of the allocated CGRA tiles. Finally, the configuration that drives the execution of multitasks on the CGRA under the new resource allocation decisions is assembled and sent to the configuration memory of each tile. This transmission is accomplished through a dedicated bus that connects each tile in a hardware-optimized sequential manner. At the software layer, a MultiSky premapper based on incremental compilation [31] is integrated with open-source front-end interpreter [32] and middle-end optimizer [33]. This integration enables comprehensive premapping of each task, ensuring that the corresponding configurations can be promptly retrieved for the CGRA to meet the stringent time constraints of dynamic resource allocation. In summary, the MultiSky framework makes the following specific contributions.

1) A hardware CGRA controller that can flexibly reallocate resources within hundreds of clock cycles, whenever multiple tasks are constructed/destructed.
2) A software CGRA premapper based on incremental compilation, which efficiently generates candidate configurations for the dynamic resource allocation scenarios of each task to avoid online transformation.
3) Extensive evaluations on real-world multitask scenarios reveal that MultiSky can enhance multitask throughput by a factor of 1.72. This is achieved by maintaining the average resource utilization at 82.7%. Additional experiments demonstrate the reasonable tradeoff between the complexity and performance of the MultiSky hardware, as well as its scalability when dealing with larger CGRA sizes and a greater number of tasks. The speed and quality of the premapping performed by the MultiSky software compiler are also quantitatively analyzed.

II. Background and Motivation

A. Working Principles of CGRAs

As depicted in Fig. 2(a), the classic CGRAs primarily consists of a tile (or processing element, PE) array. The specific size of this array is determined by the scale of the application. The well-known systolic array [37], [38], [39] is a particular subset of CGRA. In the systolic array, the PE is dedicated to performing the multiply–accumulate (MAC) operation, and the data flow is unidirectional. The tile array is

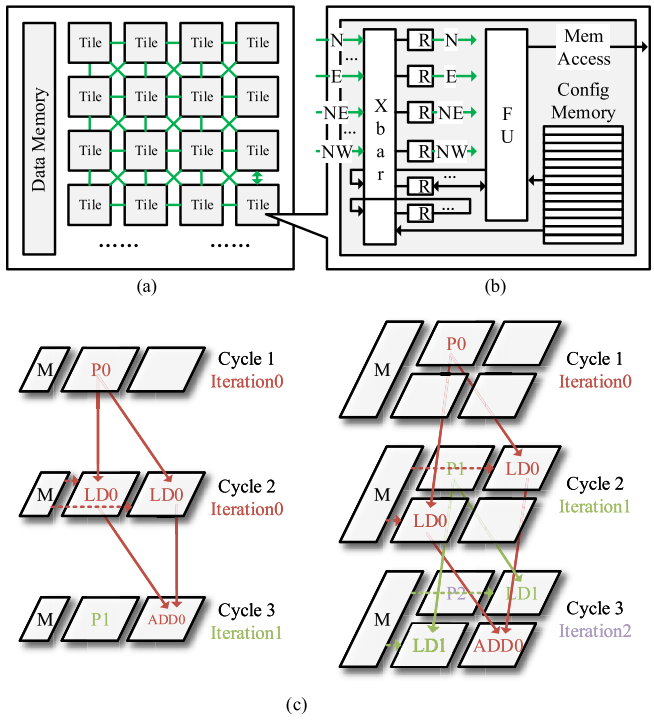

Fig. 2. Representative CGRA architecture [34], [35], [36] and principles. (a) CGRA tiles array. (b) Tile architecture. (c) CGRA is driven by configs. The placement of configuration on 2 × 1 tiles and 2 × 2 tiles are different.

equipped with a multibank data memory, which supplies the necessary data for the tile's execution.

As shown in Fig. 2(b), each tile possesses a configuration memory that controls the functions of the crossbar (Xbar) and the functional unit (FU) within the tile at every clock cycle. Through this architecture, different tiles can collaborate to achieve the target computing functionality. For instance, consider a $C = A[i]+B[i]$ that involves four operations: 1 *pointer*, 2 *load*, and 1 *add*. The execution of this task on a 2 × 1 CGRA, as arranged by the CGRA compiler [9], [10], [40], [41], [42], [43], [44], [45], [46], [47], [48], [49], is illustrated in the left part of Fig. 2(c). At cycle 1, the configuration drives tile 1 to execute the *pointer* operation. Subsequently, the result is locally registered and simultaneously transmitted to tile 2. At cycle 2, the configuration enables tile 1 and tile 2 to perform the *load* operation in parallel. After that, the results are sent and locally registered in tile 2. At cycle 3, the configuration commands tile 1 to carry out the *pointer* operation for the next iteration and tile 2 to perform the *add* operation of the current iteration.

Shifting from a 2 × 1 CGRA to a 2 × 2 CGRA can lead to improved throughput, as a new iteration can be initiated in each cycle, as shown in the right part of Fig. 2(c). Nevertheless, not only does the configuration itself change but also the placement of the config in the config memory of each tile becomes different from that in a 2 × 1 CGRA. Consequently, when the resources (tiles) for a task are dynamically adjusted, the config must be reloaded into the corresponding config memory or transformed online [28], [50] to ensure the proper functioning of the task.

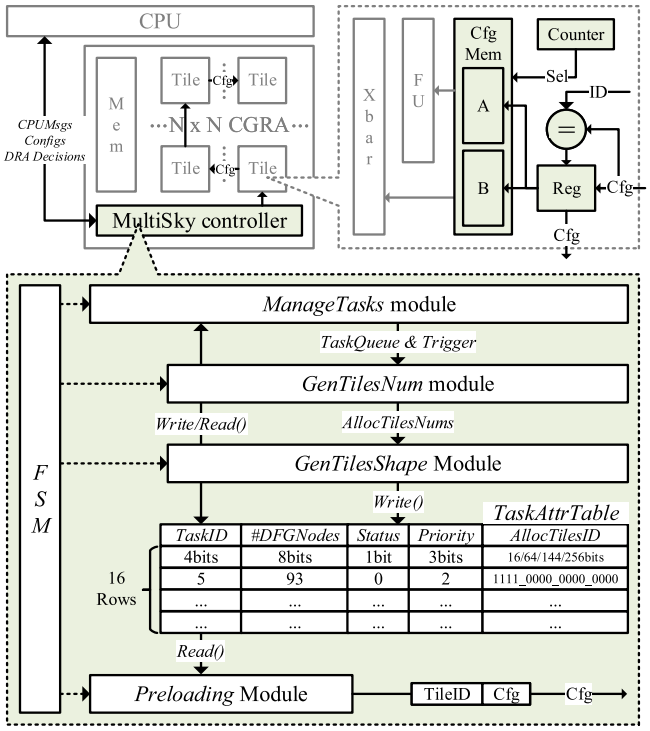

Fig. 3. Architecture of hardware MultiSky controller.

B. Motivated by the Limitations of Existing CGRA Multitask Dynamic Resource Allocation Approaches

In complex computing systems, dynamic resource allocation is indispensable [51], [52], [53]. It can precisely match resources with task requirements, avoiding resource idleness or shortages and maximizing resource utilization, thereby enhancing the reliability and performance of the entire system. Current approaches to generate decisions for CGRA multitask dynamic resource allocation can be categorized into static and dynamic.

1) Static Approaches are Inflexible and Labor-Intensive: Existing static approaches [22], [23], [24], [25], [26], [27] perform static partitioning of CGRA resources. In the spatial dimension, tiles are divided into regular long strips or squares, and in the temporal dimension, they are divided into time slices of fixed lengths. Then, engineers profile the target tasks to statically determine the spatial–temporal resource allocation for different task combinations. However, such an approach struggles to flexibly handle the random creation and destruction of runtime tasks. Take an industry portable measuring instrument MR-6000 [54] as an example. Its internal waveform calculation involves 93 types of tasks, and there could be as many as $2^{93} - 1$ combination scenarios. Profiling all these scenarios incurs an enormous manual cost and is simply unrealistic, thus restricting the flexibility of static methods.

2) Dynamic Approaches are Time-Consuming and Restricted: Dynamic approaches are time-consuming and restricted (DRIPS) [28] focuses on dynamically fine-tuning the initial resource allocation of the application, whose pipeline throughput is sensitive to the variation of input characteristics. For example, if the throughput of a specific task within the application changes with the number of input nonzeros, DRIPS uses a hardware controller to slightly adjust its resource allocation and transform the configurations of any affected tasks. However, the scale of resource changes caused by task construction/destruction is large and unpredictable, the DRIPS hardware incurs an unacceptable latency of thousands of cycles, thereby restricting the application of DRIPS in such scenarios. Amber [29] only supports dynamic resource allocation after a task finishes execution, while its follow-up research [30] further supports the task construction scenarios using a software greedy scheduler on CPU. However, the software scheme significantly restricts the frequency of dynamic resource allocation due to long latency compared to the DRIPS hardware solution.

Motivated by the above-mentioned challenges, we propose MultiSky, whose hardware controller can flexibly refresh the resource allocation decision within a few hundred cycles whenever multiple tasks are constructed/destructed. This mechanism completely eliminates the requirements for manual profiling, and can be called frequently due to low latency. Besides, the software MultiSky premapper statically generates the configuration candidates through an incremental way [31], which further avoids the latency introduced by online config transformation [28], [50] after the dynamic resource allocation.

III. Hardware MultiSky Controller

Fig. 3 shows the hardware architecture of the MultiSky controller, which is composed of a *TaskAttrTable*, *ManageTasks* module, *GenTilesNum* module, *GenTilesShape* module, a finite state machine (FSM), and the configuration *preloading* module with the corresponding receiver distributed in each tile that are connected through a dedicated bus. The design details are discussed below.

A. TaskAttrTable

TaskAttrTable is either a RAM or register file to record attributions of each task. The fields include *TaskID* (8 bits): support a wide range of Task identification 0∼255, which is sufficient for most of scenarios; *#DFGNodes* (8 bits): support a maximum of 256 nodes in task dataflow graph (DFG), as most of the embedded kernels are not that complex; *Status* (1 bit): whether the task has finished execution, either run 1, or done 0; *Priority* (3 bits): the task priority that affects the number of allocated tiles, supports the range of priority from 0∼7. The priority of each task is automatically determined by the CPU and affected by user behaviors; *AllocTilesID* (16 bits for 4 × 4 CGRA, 64 bits for 8 × 8 CGRA, 144 bits for 12 × 12 CGRA, 256 bits for 16 × 16 CGRA): the ID of allocated tiles to each task, utilizes one-hot encoding to represent the allocated tiles of the current task. For example, *AllocateTilesID* = 1111_0000_0000_0000, with the 1st, 2nd, 3rd, and 4th bits set to 1, representing corresponding task is allocated with Tile 0, 1, 2, and 3. *TaskAttrTable* will be accessed by all other modules.

To control the hardware overhead, the number of rows in *TaskAttrTable* is fixed to 16, which means MultiSky can

TABLE I
SIZE OF THE TASK ATTRIBUTES TABLE

CGRA size	#Rows	Row sizes(bits)	Table sizes(Bytes)
4×4	16	32	64
8×8	16	80	160
12×12	16	160	320
16×16	16	272	544

Algorithm 1: *ManageTasks* Module

Input: *CPUMsgs*; *TaskAttrTable*;
Output: *TaskQueue*; *Trigger*;

```
1  foreach Msg in CPUMsgs do
2      switch Msg.Action do
3          case Construction:
4              TaskAttrTable.Write(Msg.TaskAttrs);
5              Trigger=True;
6          case Destruction or Task.isDone():
7              TaskAttrTable.Write(Msg.TaskID,
8              Status = 0);
9              Trigger=True;
10
11     endsw
12 end
13 foreach TaskID do
14     if TaskAttrTable.Read(Msg.TaskID, Status) then
15         TaskQueue.push(TaskID);
16     end
17 end
```

support 16 parallel tasks at maximum, no matter what CGRA sizes are adopted or how many tasks we have. When a new *CPUMsgs* comes to construct a new task, the controller will retrieve the *Status* field of 16 rows, and write the new task attributes to a specific row if the *Status* = 0. The size of *TaskAttrTable* when CGRA size increases is shown in Table I, with only 544 bytes under 16 × 16 CGRA.

B. ManageTasks Module

Algorithm 1 illustrates the principles of the *ManageTasks* module. The input contains *CPUMsgs* from CPU and *TaskAttrTable*. The output is the refreshed *TaskQueue* and the boolean *Trigger* that indicates whether dynamic resource allocation should be triggered. The functionality of *ManageTasks* module is to maintain the *TaskAttrTable* and generate corresponding outputs. For each message within *CPUMsgs*, if the action is task *Construction*, all the attributes of this task will be written (*Write*()) to a line in *TaskAttrTable* that has *Status* marked with 0. If the action is task *Destruction* or finished execution (*Task.IsDone*()), the *Status* field of a specific line in *TaskAttrTable* identified by *TaskID* is set to 0. *Trigger* will be set to *True* whenever there are new tasks constructed, force-destructed, or finished execution. After all these are done, *TaskID* whose *Status* is 1 will be extracted to the *TaskQueue*.

Algorithm 2: *GenTilesNum* Module

Input: *TaskQueue*; *Trigger*; *TaskAttrTable*; *TotalTiles*;
Output: *AllocTilesNums*;

```
1  if !Trigger then
2      Exit;
3  end
4  TotalDFGNodes = CalTotalNodes(TaskQueue,
   TaskAttrTable),
5  foreach TaskID in TaskQueue do
6      P = TaskAttrTable.Read(Msg.TaskID, Priority);
7      NumTiles = #DFGNodes*P / ∑TotalDFGNodes * TotalTiles;
8      TheoryMax = ⌈#DFGNodes / II⌉;
9      NumTiles = min(NumTiles, TheoryMax);
10     AllocTilesNums.push(NumTiles);
11 end
```

C. GenTilesNum Module

Apart from the output of the *ManageTasks* module, including *TaskQueue* and *Trigger*, the *GenTilesNum* module also takes *TaskAttrTable* and the total number of CGRA tiles *TotalTiles* as inputs, finally outputs the allocated tiles for each task within the *TaskQueue*. As shown in Algorithm 2, if *Trigger* is true, the module will generate the new resource allocation decision. For each task within the *TaskQueue*, the priority *P* of current *TaskID* is first queried from *TaskAttrTable* (*Read*()). Then the number of allocated tiles *NumTiles* is calculated by the weighting average (line 7). It takes the proportion ($\#DFGNodes * P / \sum TotalDFGNodes$) as weights, where *#DFGNodes* is the number of DFG nodes of the current task, and *TotalDFGNodes* is the sum of DFG nodes for all tasks in *TaskQueue*. Besides, *NumTiles* is further regulated using the theoretical maximum value *TheoryMax* (*II* is calculated using formula in [55]) before pushing into *AllocTilesNums* (lines 8–10), because the additional tiles that exceed *TheoryMax* will be wasted by the CGRA mapper.

D. GenTilesShape Module

Algorithm 2 only provides the number of tiles allocated for each task. There should be a shape generation algorithm to further decide the shape of allocated tiles, so that all shapes of different tasks correctly form the shape of the CGRA tiles array. As the "dense" shape provides more abundant routing resources than the "sparse" shape, the CGRA compiler has the potential to achieve higher task throughput (discussed in detail in Section IV). Therefore, an ideal shape generation algorithm should provide dense shapes for allocated tiles of each task as much as possible, so as to maximize the throughput of all tasks. However, the target above belongs to a combinatorial problem, whose computational complexity increases magnitude as the size of CGRA and number of tasks increase, which does not satisfy the real-time scheduling requirements and has significant hardware overhead.

To tradeoff shape's routing resource density and time overhead, MultiSky proposes a heuristic Algorithm 3 for *GenTilesShape* module based on coordinates. For each task

Algorithm 3: *GenTilesShape* Module

Input: *TaskQueue*; *AllocTilesNums*; *TaskAttrTable*;

```
1  X = 0; Y = 0;
2  foreach TaskID in TaskQueue do
3      NumTiles = AllocTilesNums[TaskID];
4      while NumTiles > 0 do
5          (H, V) = QueryXYStrides(NumTiles);
6          foreach (h, v) in (H, V) do
7              if Legal((X, Y), (h, v)) then
8                  AllocTilesID = Find((X, Y), (h, v));
9                  X+ = h; Y+ = v;
10                 TaskAttrTable.Write(Msg.TaskID,
                   AllocTilesID);
11                 NextTask();
12             end
13         end
14         NumTiles − −;
15     end
16 end
```

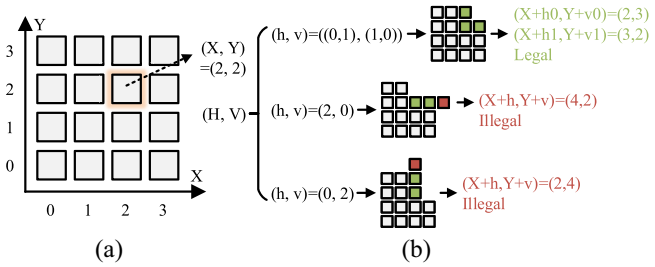

Fig. 4. Example to explain (h,v) in Algorithm 3. (a) 4 × 4 CGRA coordinates. (b) Predefined (H, V) under NumTile = 3.

in *TaskQueue*, the number of allocated tiles *NumTiles* is first fetched from *AllocTilesNums* (line 3), then the vertical and horizontal length (H, V) of the predefined shapes under current *NumTiles* are queried using *QueryXYStrides*(*NumTiles*) (line 5). There might be multiple (h, v) within one (H, V) to represent different predefined shapes. Then the algorithm retrieves each stride (h, v) within (H, V), and checks whether current coordinates (X, Y) are still legal after adding with (h, v), including not reaching the CGRA border (lines 6 and 7). If it is legal, then all tiles with the range of (h, v) will be found (*Find*()) and wrote (*Write*()) back to *TaskAttrTable* and (X,Y) is updated (lines 8–10), which marks that the shape of allocated tiles for current task is generated. The algorithm will then move to the next task (line 11). Otherwise, *NumTiles* will be decreased (line 14) to loose the constraint and try again.

Fig. 4 provides a detailed workflow example to illustrate Algorithm 3. Fig. 4(a) illustrates the coordinate system of a 4 × 4 CGRA, allowing 16 Tiles to be located using (X, Y). Assuming the current task has *NumTiles* = 3 with $(X, Y) = (2, 2)$, Fig. 4(b) shows three pairs of predefined (h, v) contained in (H, V). The first includes the nested tuples $(h, v) = ((0, 1), (1, 0))$, representing an nonrectangle dense shape. The remaining pairs are $(h, v) = (2, 0)$ and $(h, v) = (0, 2)$, corresponding sparse shapes of different directions,

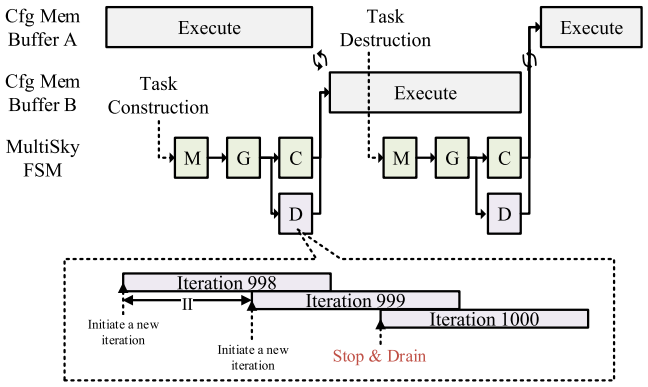

Fig. 5. Workflow of MultiSky hardware controller managed by FSM.

respectively. Line 6 of Algorithm 3 first retrieves the pair $(h, v) = ((0, 1), (1, 0))$ for legality checking. Since both $(X + h_0, Y + v_0) = (2, 3)$ and $(X + h_1, Y + v_1) = (3, 2)$ fall within the coordinate range shown in Fig. 4(a) and correspond to physical Tiles, the *Legal*() check at line 7 passes. Tiles at coordinates (X, Y), $(X + h_0, Y + v_0)$, and $(X + h_1, Y + v_1)$, specifically $(2, 2)$, $(2, 3)$, and $(3, 2)$, are located (line 8) and written into the *TaskAttrTable* (line 10). Simultaneously, (X, Y) is updated to $(X + h1, Y + v0) = (3, 3)$ at line 9, marking the completion of the current task's resource allocation, so that no further checks are performed on the other (h, v) pairs. It should be noted, however, that these remaining pairs also cannot pass the legality check, as coordinates resulting from $(X + h, Y + v)$, specifically $(4, 2)$ and $(2, 4)$, fall outside the range in Fig. 4(a) and represent nonexisting Tiles, therefore, marked as illegal in Fig. 4(b).

E. Preloading Module and Buses

As discussed in Section II-A, task execution on CGRA is driven by configuration, whose placement on the tile's config memory is completely different under various tile shapes. Therefore, the configuration for each tile must be reloaded after dynamic resource allocation that changes the shapes of allocated tiles. As shown in Fig. 3, to hide the latency of reloading, the config memory of each tile is designed as a double buffer. The config sent from the CPU will be first supplemented with the correct *TileID* by the preloading module, then sent to the first tile through the designated bus. Each tile will always register the received config, and then send it to the next tile. If the *TileID* field of the registered config is equal to local *TileID*, they are written to one of the buffers in config memory as controlled by a 1-bit counter. Such a design is hardware-friendly and can improve timing.

F. FSM

The FSM controls the working sequence of the four functional modules discussed above. As visualized in Fig. 5, every time after dynamic resource allocation by the task construction/destruction, FSM iteratively enables the ManageTasks module (M), GenTilesNum and GenTilesShape module (G) to generate new resource allocation decisions and record in the Task Attributes table. Then informs the CPU to initiate the

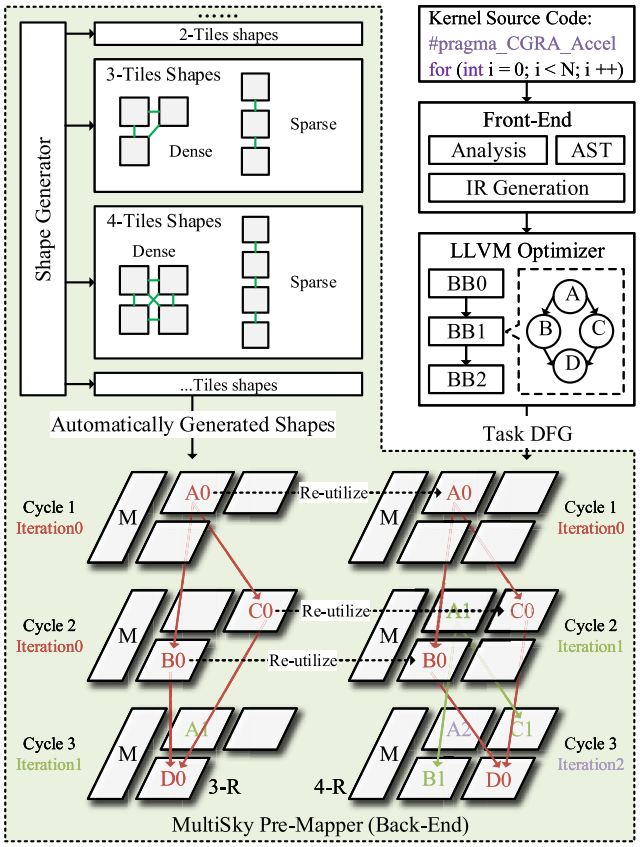

Fig. 6. Composition of software MultiSky premapper.

ConfigPreloading module (C) that transmits configurations of the new resource allocation to another config memory buffer. Simultaneously, Drain module (D) notices the tiles that mapped the first node of each task's DFG to stop initiating new iterations and let all initiated iterations finish to consume the intermediate results with Tile registers. Finally, triggers CGRA to switch the config memory buffer so that CGRA execute under the new resource allocation decision. FSM achieves seamless dynamic resource allocation by overlapping config preloading with CGRA execution.

G. Other Details

MultiSky requires the memory hierarchy of CGRA should be similar to HReA [56], where number of memory banks equals to the number of tiles (PEs). And each tile can access any banks using a request arbiter. In such case, MultiSky ensure that tasks of any number of memory access operations can be correctly mapped on to the dynamically allocated tiles.

IV. Software MultiSky Premapper

Fig. 6 shows the compilation framework of MultiSky. As dynamic resource allocation has strict time constraints to minimize the impact on the task throughput, the corresponding configurations of different resource allocation decisions are not appropriate to be transformed online like [28], [50]. Therefore, the MultiSky compiler chooses to perform comprehensive offline premapping and store the configurations in the off-chip

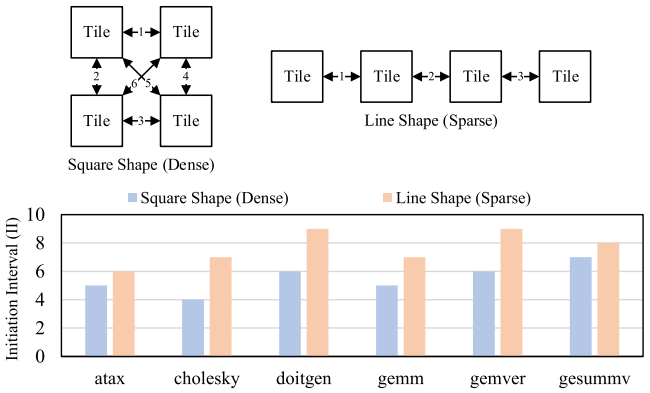

Fig. 7. Dense and sparse shapes, and how they affect the mapping performance. Results are obtained using open-source CGRA mapper [34].

memory, so that they can be transmitted to CGRA through DMA each time after dynamic resource allocation.

MultiSky premapper is established based on the C/C++ front-end and LLVM optimizer. Developer should first use performance analysis tools to locate the bottleneck task of the application, then add directive *#pragma_CGRA_Accel* to the task (i.e., a for loop), so that the front-end (e.g., Clang for C/C++ [32]) can automatically identify the acceleration task. Second, the front end performs lexical and syntactic analysis to create an abstract syntax tree (AST) from the task source code. Third, the LLVM optimizer [33] turns the AST into an intermediate representation (IR) and optimizes it. Subsequently, LLVM extracts basic blocks from the optimized IR, analyzes instruction dependencies, and finally creates nodes and edges to form a DFG.

MultiSky premapper takes the task DFG as one of the inputs, while another input is the automatically generated resource allocation decisions. Given a CGRA with $N \times N$ tiles array, the possible allocated number of tiles for any task varies from $0 \sim N^2$. For each number of allocated tiles, the shape generator algorithm generates two dense and sparse shapes that are consistent with Algorithm 3. The DFG of each task will be premapped on all generated shapes and the corresponding configurations candidates will be stored in the off-chip memory. For efficient premapping, MultiSky adopts the incremental CGRA mapper [31], which reutilizes the generated configs of the previous shape to dramatically accelerate the mapping speed of the current shape with negligible mapping quality loss.

The classification of "dense shape" and "sparse shape" depends on how many routing resources (CGRA Links) the shape provides. Under a specific *NumTiles* in Algorithm 3, the shape with more routing resources is defined as a dense shape, the shape with less routing resources is defined as a sparse shape. For example, in Fig. 7, the Square shape has six Links and the Line shape has three Links, therefore the former is dense and the latter is sparse. The reason why it is defined so counter-intuitive is that we hope the dense shape should bring a lower initiation interval (II) (higher throughput) for CGRA mapping than the sparse shape. As shown in Fig. 7, for six tasks from polybench [57], the square shape (dense)

TABLE II
INDUSTRIAL BENCHMARK: MECOBENCH

Apps	Tasks	Predecessors
Frequency analysis to torque & vibration signals	FFT_T: FFT to torque signal	N/A
	PS_T: Power spectrum to FFT_T	FFT_T
	RMS_T: Root mean square to FFT_T	FFT_T
	FFT_V: FFT to vibration signal	N/A
	PS_V: Power spectrum to FFT_V	FFT_V
	RMS_V: Root mean square to FFT_V	FFT_V
Motor power & efficency analysis	PIN: Inverter output power	N/A
	POUT: Motor output power	N/A
	EFF: Motor efficiency	PIN&POUT
XY analysis	AREA_PIN: Area of XY plotting to PIN	PIN
	ANG_PIN: Angle of XY-plot to PIN	PIN
	AREA_POUT: Area of XY-plot to POUT	POUT
	ANG_POUT: Angle of XY-plot to POUT	POUT
	AREA_EFF: Area of XY-plot to EFF	EFF
	ANG_EFF: Angle of XY-plot to EFF	EFF
	AREA_T: Area of XY-plot to torque	N/A
	ANG_T: Angle of XY-plot to torque	N/A

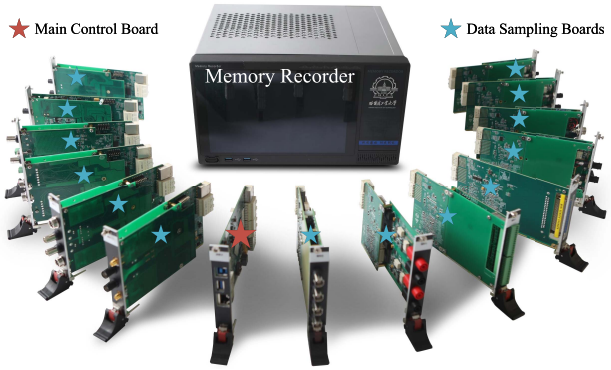

Fig. 8. Self-developed memory recorder with 14 data sampling boards and one main control board.

average has 2.17 lower II than the line shape (sparse) because of more CGRA links. Therefore, for each *NumTiles*, the sparse shape is the alternative option if the dense shape is illegal at Algorithm 3 line 7.

V. EXPERIMENTAL SETUP

A. Implementation and Deployment

Memory recorder is a representative multitask system in the test and measurement domain. Fig. 8 shows our self-developed memory recorder, with multiple data sampling boards and a main control board to serve the purpose of capturing, storing, and retrieving multiple types of sensor data, for later review, analysis, or record-keeping. A 200 MHz 4 × 4 CGRA with MultiSky controller is implemented using Verilog and deployed on the AMD Kintex-7 FPGA [58] in the main control board to accelerate the data analysis. The MultiSky premapper is deployed on the 12th Gen Intel i7 CPU [59].

B. Baseline

This article takes representative static and dynamic resource allocation decision generating approaches as baselines to make a robust evaluation of MultiSky. The static baseline is the classic TRIPS [22], under the premise of unpredictable task quantity and complexity, the 4 × 4 CGRA is spatially divided into 16 groups, and temporally divided into every 1000 clock

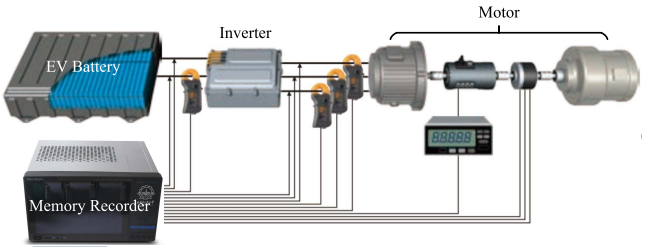

Fig. 9. Schematic of using a memory recorder to analyze the dynamic characteristics of a motor.

cycles as a time stamp. The dynamic baseline is the state-of-the-art DRIPS [28]. As DRIPS is initially designed for dynamic resource allocation of the streaming tasks whose pipeline is sensitive to the input characteristic variation, its performance on MeCoBench introduced in Section V-C might be limited. Therefore, the DRIPS baseline is redeveloped to deal with the random task construction and destruction using its core mechanisms, including *downgrade*(), *upgrade*(), and *reshape*(), when performing experiments on MeCoBench. When conducting experiments on StreamingBench [60] introduced in Section V-D, DRIPS maintains the original design.

C. Industrial Benchmark: MeCoBench

A real-world multitask benchmark is extracted from an industry memory recorder manual [54], namely, MeCoBench. As shown in Fig. 9, we use the memory recorder to measure the dynamic motor characteristics, i.e., record the fluctuations in various parameters from the motor's start to stop, more details are revealed in Table II.

The first task is the simultaneous frequency analysis of the measured motor torque and vibration signals using fast Fourier transform (FFT), thereby identifying the abnormal signals during motor running. The power spectrum (PS) and root-mean-square spectrum (RMS) will be constructed immediately after FFT is done, as their calculation relies on the results of FFT. There are six tasks in the first application. The second task is the motor power and efficiency analysis which are crucial to the final product quality. The inverter is responsible for transferring the direct current to the alternating current accepted by the motor, thus the output power (POUT) of the inverter based on the measured voltage and current serves as the input power (PIN) of the motor. The POUT is calculated based on the measured torque and rotation signals. As soon as PIN and POUT finish, the motor efficiency (EFF) can be initiated for execution. The third task is the XY analysis of the torque signal, and the results of PIN, POUT, and EFF. The XY analysis includes the calculation area (AREA) and angle (ANG) to analyze correlation and evaluate trends of change.

Noted that for the 17 tasks within three applications, tasks with no *predecessors* will be constructed simultaneously at the beginning of multitask runtime, tasks with *predecessors* will be constructed automatically only after their predecessors finish execution, and all tasks will be destructed immediately after done. The input length of all tasks is fixed to 4096 sampled points as it is the default memory recorder screen displaying length.

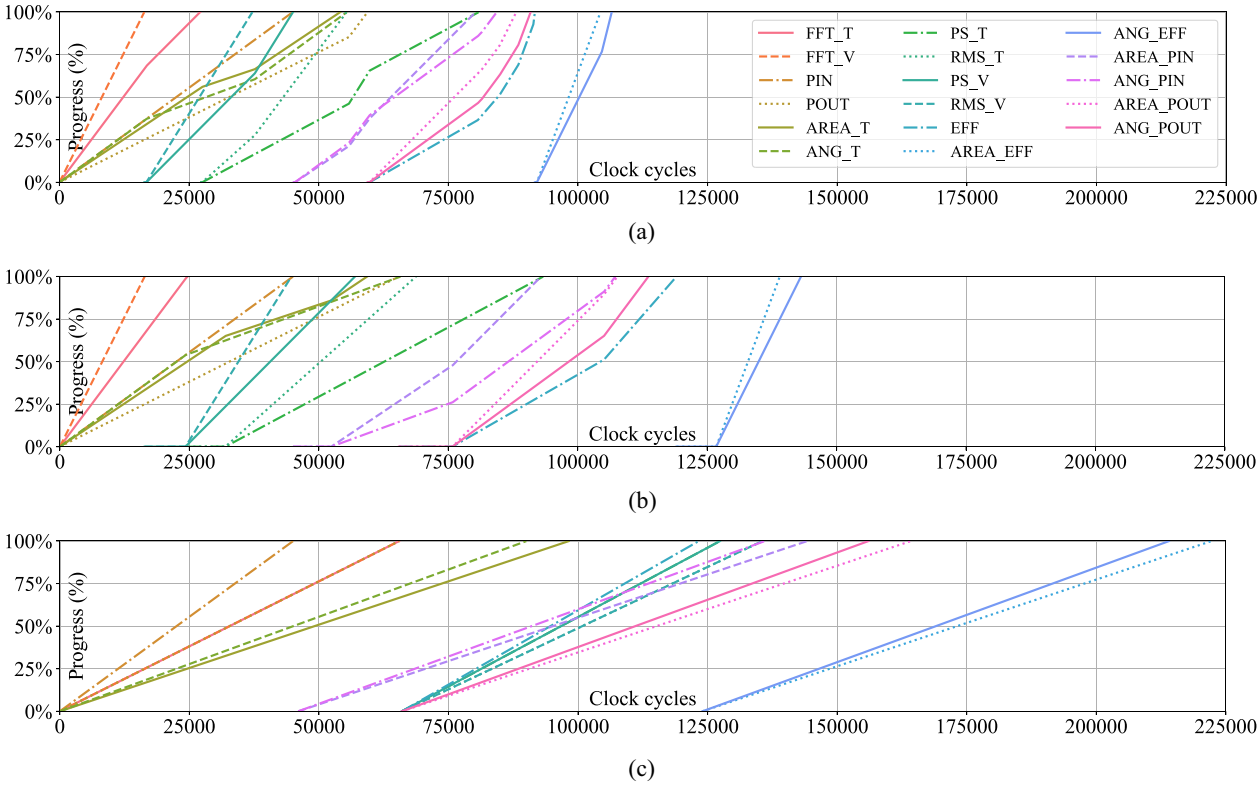

Fig. 10. Progress of each task within MeCoBench changes over time under 4 × 4 CGRA, with different multitask dynamic resource allocation methods. (a) MultiSky flexibly allocates resources during multitask run-time, and spends 106 516 cycles to finish all tasks. (b) DRIPS works slowly when generating resource allocation decisions for task cons/destruction, and takes 1 43 114 cycles to finish all tasks. (c) TRIPS is inflexible and spends 222 205 cycles to finish all tasks.

TABLE III
ACADEMIC BENCHMARK: STREAMINGBENCH

Apps	Tasks	Predecessors
Ray Tracing (RayTracing)	CreateRay: Ray initiation	N_A
	PrioObj: Adjust interact priority	N_A
	TrackInt: Track interaction	CreateRay
	DetPos: Intersection coordinates	TrackInt
	CaptureRay: Ray interaction to scene	DetPos
	GetColor: Intersection colors	CaptureRay
	SetColor: Write frame buffer	GetColor
Graphic Convolutional Networks (GCN)	Compress: Input compression	N/A
	Aggregate1: Collect neighbor info	Compress
	CombRelu: Combine with ReLU	Aggregate1
	Aggregate2: Collect neighbor info	CombRelu
	Combine: Generate new features	Aggregate2
	Pooling: Dimension reduction	Combine
Lower-Upper Decomposition (LU)	Init: Initiate variables	N/A
	Decompose: Decompose matrix	Init
	Solver0: Solving linear equations	Decompose
	Solver1: Solving linear equations	Decompose
	Invert: Inverse of matrix	Decompose
	Determinant: Determinant of matrix	Decompose

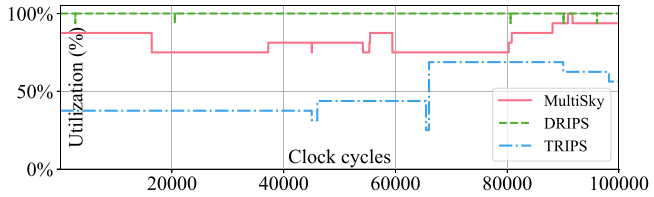

Fig. 11. Tile utilization of 4 × 4 CGRA under different CGRA resource allocation schemes during multitask runtime.

D. Academic Benchmark: StreamingBench

To further demonstrate MultiSky's throughput advantages on applications with strong task dependencies, we also perform experiments on the StreamingBench [60] open-sourced by DRIPS. As shown in Table III, StreamingBench includes graph convolutional networks (GCNs, from AI domain), RayTracing (Ray tracing, from graphic rendering domain), and LU (Lower-upper decomposition, from linear algebra domain). Each application contains 6∼7 tasks that execute in a pipeline manner to process the continuous input of data. For example, a two-layer GCN for enzyme classification has the following data dependencies: The continuous input enzyme graph data are first sent through a compression task (Compress), then fed the compressed data to the first layer, which integrates an aggregation task (Aggregate1) and a fused task (CombineReLU). The output of the first layer is continuously processed by the second layer, which consists of a second aggregation task (Aggregate2), a standalone task (Combine), and culminates in a task (Pooling) that generates the final graph-level representation for classification.

As tasks within each application in StreamingBench have strong data dependencies and execute in a pipeline manner to process streaming inputs, both construction and destruction should be done on a per-application basis. For example, all tasks in GCN are constructed or destructed simultaneously on CGRA, the separate construction or destruction of any single

task in GCN will break the pipeline, and result in the entire application being unable to process input data anymore.

VI. Experimental Results

A. Throughput Improvement on MeCoBench

Fig. 10 displays the progress of each task within MeCoBench changes over time under 4 × 4 CGRA, with the dynamic resources allocation powered by MultiSky, DRIPS, and TRIPS. MultiSky spends 106 516 cycles to finish all tasks, whose throughput is 1.34 × and 2.09 × higher than DRIPS and TRIPS, respectively.

Such a performance improvement benefits from MultiSky's flexibility and efficiency to dynamically allocate resource whenever there are new tasks constructed or destructed. As shown in Fig. 10(a), the slope of the progress curve for each task frequently changes during its execution. Resources are absorbed from the executing task if there are new tasks constructed, therefore the slope of the progress curve of the existing task decreases. If the task finishes execution and releases its resources, all other running tasks reutilize those resources to speed up, so that the slope of the corresponding progress curve increases.

DRIPS can only fine-tune the initial resource allocation decision made at the beginning using *downgrade*() or *upgrade*(), increase or decrease one CGRA tile at each time, and *reshape*() any tasks whose shape of allocated tiles are affected. However, when a task is constructed or destructed, many tiles are reallocated, which makes the fine-tune mechanism introduce tremendous time overhead because *downgrade*(), *upgrade*(), and *reshape*() are called repeatedly. Consequently, DRIPS spends an additional 36 598 clock cycles to finish all tasks.

TRIPS spatially partitions the CGRA tiles to 16 groups, while there are only a maximum of ten tasks run parallelly, which means six groups of tiles are unutilized during multitask run-time. Besides, the time stamp of 1000 cycles is so sparse that the tiles released after task destruction can not be reutilized during this period. As the result shown in Fig. 11, the tile utilization of TRIPS varies from only 25% to 62.5%, which is the key reason why TRIPS spend more than twice time than MultiSky to finish all tasks. One may argue that such a performance gap is due to unreasonable spatial and temporal partition parameters of TRIPS. However, the multitask scenario is unknown to TRIPS, as well as the random task construction or destruction during run-time, which makes it impossible for TRIPS to predefine a suitable partition strategy.

Fig. 11 also demonstrates that a over-fine-grained resource allocation strategy may harm the multitask throughput by contrast. Although DRIPS achieves almost 100% tile utilization throughout the multitask run-time by using a fine-grained strategy, including *downgrade*(), *upgrade*(), and *reshape*(), the multitask throughput is decreased because of the significant time overhead of DRIPS. MultiSky, however, sacrifices the utilization to an average 82.7% to finish the resource allocation as soon as possible, thereby achieving even higher multitask throughput compared with the average 99.99% utilization of DRIPS.

TABLE IV
Hardware Overhead That MultiSky Introduces to CGRA With Different Sizes

	4×4 CGRA	8×8 CGRA	12×12 CGRA	16×16 CGRA
DSP	16.67%	4.17%	1.85%	1.04%
FF	9.68%	2.55%	1.17%	1.18%
LUT	11.67%	2.86%	0.76%	0.79%

B. Throughput Improvement on StreamingBench

Fig. 12 displays the progress of each application within StreamingBench changes over time under 12 × 12 CGRA, with the dynamic resources allocation powered by MultiSky and DRIPS, respectively. Three applications are constructed simultaneously onto the CGRA, with equal 4 × 12 tiles allocated to each. RayTracing has the most input data, so its execution time is the longest. GCN and LU have fewer inputs and are destructed separately after done. DRIPS reduces pipeline bubbles by fine-tuning resource allocation, while MultiSky cannot. Therefore, DRIPS completes the GCN application 4.03 s earlier than MultiSky. But after the destruction of GCN, DRIPS is unable to utilize the 4 × 12 idle tiles, and MultiSky reallocates the idle tiles to LU and RayTracing, which gradually catch up with DRIPS's progress and almost completes the LU application at the same time. After the destruction of LU, DRIPS is unable to utilize the released idle resources, while MultiSky is able to, ultimately leading MultiSky to complete the RayTracing application 6.67 s ahead of DRIPS. Overall, although MultiSky does not have the ability to fine-tune the internal pipeline of the application, its high sensitivity to resource release and competition gives it throughput advantages compared to DRIPS under multitask construction and destruction scenarios.

C. Hardware and Time Overhead

Table IV shows the additional hardware overhead that MultiSky brings to CGRA. As both of them are deployed on FPGA, the overhead is evaluated using the FPGA resources, including DSP, FF, and LUT. The implementation of MultiSky requires three DSPs, 1994 FFs, and 4201 LUTs. It uses the register file composed of FF instead of BRAM to implement the *TaskAttributesTable*, thereby achieving higher access speed. DSP is used mainly in the *GenTilesNum* module. LUT is used for supporting combinational logic for FSM and all modules within MultiSky. On average, the MultiSky introduces additional 12.67% hardware resources to a 4 × 4 CGRA, this figure further decreases to 3.19%, 1.26%, and 1.00% when CGRA sizes increase to 8 × 8, 12 × 12, and 16 × 16, respectively. This demonstrates that the hardware overhead of MultiSky decreases with the increases in CGRA size.

Table V further shows the time overhead of MultiSky. Increasing the number of tasks does not significantly increase the time overhead, because lines 6–10 of Algorithm 2 and lines 5–7 and 8–10 of Algorithm 3 can both be pipelined between

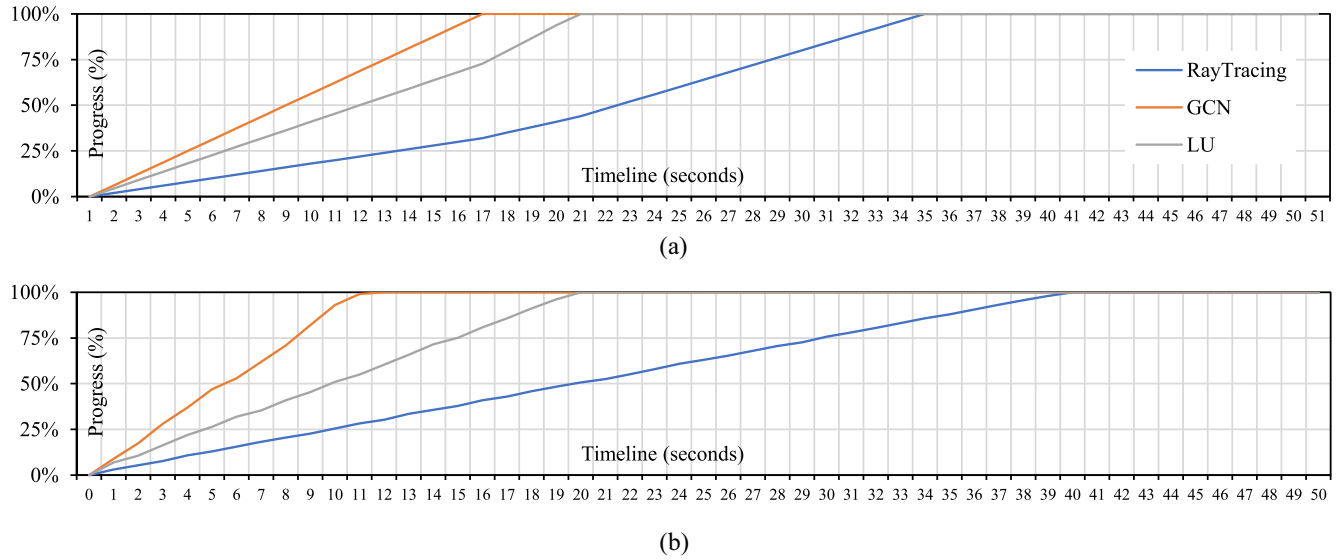

Fig. 12. Progress of each application within StreamingBench changes over time under 12 × 12 CGRA, with different multitask dynamic resource allocation methods. (a) MultiSky flexibly utilizes idle resources after GCN and LU finish execution, and spends 33.21 s cycles to finish RayTracing. (b) DRIPS leaves most of tiles idle after GCN and LU destructed, and takes 39.88 s to finish RayTracing.

TABLE V
EXECUTION CLOCK CYCLES OF MULTISKY UNDER DIFFERENT CGRA SIZES AND NUMBER OF TASKS

	4×4 CGRA	8×8 CGRA	12×12 CGRA	16×16 CGRA
4 Tasks	204	323	408	518
8 Tasks	223	337	419	523
12 Tasks	246	348	426	537
16 Tasks	257	363	445	551

adjacent tasks. When the size of CGRA increases from 4 × 4 to 16 × 16, the time overhead increases by more than 300 cycles, because the number of tiles has increased by 16 times, and lines 8–10 of Algorithm 3 requires more cycles to finish. However, hundreds of cycles are negligible compared with task length, especially when it is perfectly overlapped with the task execution as shown in Fig. 5.

D. Compiler Performance

Fig. 13 shows the premapping time consumption and mapping quality of each task using the MultiSky premapper. Under 4 × 4 CGRA, the premapping time for each task is the sum of the time required to map the task on 16 dense shapes and 16 sparse shapes, and premapping all eight tasks takes 27.52 s in total. The total premapping time increases to only 440.32 s under 16 × 16 CGRA, thanks to the reutilization mechanism in the incremental compilation mechanism. Besides, the MultiSky premapper can provide an average of 96.6% of optimal mapping quality (the minimum possible initial interval) for all eight tasks.

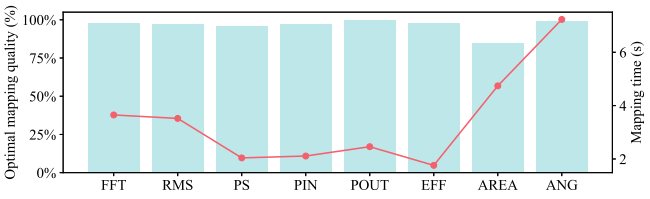

Fig. 13. Premapping quality (bar) and premapping time consumption (curve) for each task within the multitask scenario.

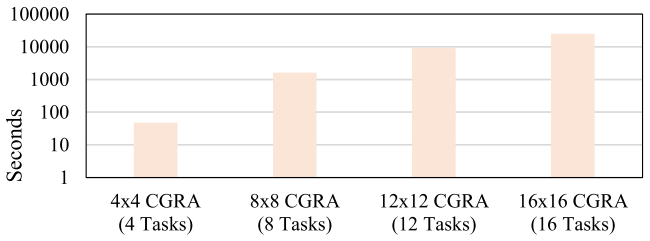

Fig. 14. Time consumption required for *Q*-learning-based MultiSky controller to converge under different CGRA sizes and number of tasks.

VII. DISCUSSION

A. Scalability Issue of Machine Learning-Based MultiSky Controller

The machine learning (ML) method suffers from long inference latency; to mitigate this, we employed *Q*-learning [61], a representative ML method for fine-tuning allocation decisions, during the earlier motivation stage of MultiSky to discover the global optimal solution. The *Q*-learning *Agent* autonomously allocates CGRA tiles to different tasks, with *States* encoding tile utilization, task priorities, and shape continuity; *Actions* span tile-selection initiation, directional expansion (up/down/left/right), and allocation confirmation; *Rewards* combine shape compactness and adjacency bonuses while penalizing fragmentation; *Evaluation metrics* measure shape density (density/sparse scores), contiguous block integrity, etc. However, experimental results in Fig. 14 still demonstrate the scalability issue of *Q*-learning. It performs acceptably on 4 × 4 CGRA with four tasks, with high tile utilization and dense shapes, but requires 7.5 hours to converge when CGRA size increases to 16 × 16 and the number of tasks increases to 16.

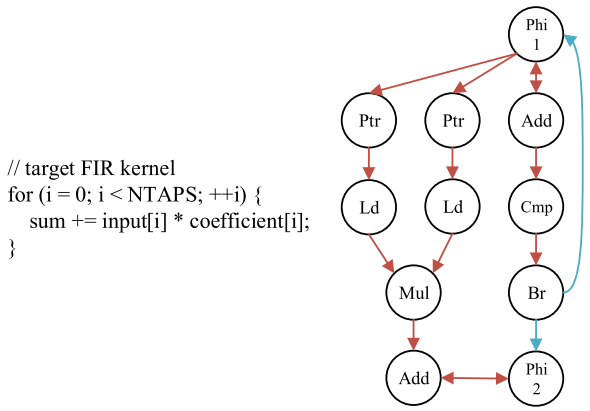

Fig. 15. Control flow of the FIR task is transferred to specific DFG nodes by MultiSky premapper.

The poor scalability of the *Q*-learning method limits its deployment on the CGRA multitask dynamic resource allocation, which may be triggered once every few seconds and leaves no time for *Q*-learning's slow convergence (hours). Therefore, MultiSky finally proposes a heuristic shape generation algorithm, which aims to tradeoff between allocation decision quality and overhead. Now it can find a suboptimal resource allocation decision (average 82.7% tile utilization) with a 200∼500 clock cycles execution time overhead and negligible hardware overhead, which is lightweight enough to support the frequent CGRA multitask dynamic resource allocation.

B. How MultiSky Premapper Handles Control Flow

The CGRA architecture that MultiSky targets for and the MultiSky premapper explicitly support the control flow using branch predication [62]. For example, the finite impulse response (FIR) task is transferred to DFG shown in Fig. 15, where the hidden control flow in the loop $for(i = 0; i < \text{NTAPS}; + + i)$ are automatically transferred into blue DFGEdges and DFGNodes, including *Phi*1, *Phi*2, *Add*, *Cmp*, and *Br*. If $i <$ NTAPS, *Phi*1 will pass variable i to *Ptr*, and *Phi*2 will pass variable sum back to Add to achieve accumulation. Otherwise, *Phi*1 and *Phi*2 pass nothing. However, significant challenges remain in achieving CGRA acceleration for certain complex control flows, such as dynamic loop bounds and nonperfectly nested loops. Programmers should avoid adding acceleration statements (#pragma_CGRA_Accel) to those loops.

C. TaskQueue Traverse Order

Algorithm 3 traverses *TaskQueue* in a sequential order, which makes the task in the tail of the queue more likely generate the sparse shape that degrades the throughput. We have calculated the II growth caused by assigning sparse shapes in different multitasking scenarios. As shown in Table VI, average 11.46% tasks have II increment by average 0.34 due to sparse shape. At the earlier stage of MultiSky, we once establish a compensation mechanism that the task has sparse shape in the current round will be fetched from the queue in advance in the next round, so as to generate a dense shape. However, such a mechanism requires the sorting to the *TaskQueue* and will increase the controller execution cycles for around 15.3%. And it does not fundamentally solve the problem that tail tasks will always have sparse shapes. Therefore, the simple sequential traverse order is adopted.

TABLE VI
PERFORMANCE DEGRADATION (16 × 16 CGRA) CAUSED BY SEQUENTIAL TRAVERSE ORDER TO *TaskQueue*

#Tasks	4	8	12	16	Avg
Avg Optimal II	4	5.63	6.83	9.56	6.51
#Sparse Shape	0%	12.50%	8.33%	25.00%	11.46%
Avg Actual II	4	6	7.08	10.31	6.85

VIII. CONCLUSION

MultiSky advances CGRA-based multitask systems by enabling efficient and flexible dynamic resource allocation. Its hardware-software co-design achieves three key contributions: 1) a hardware controller that reallocates resources within hundreds of cycles; 2) a premapper that eliminates online configuration latency; and 3) a systematic evaluation demonstrating 1.72× throughput gains over state-of-the-art approaches. The framework's low hardware overhead (1% for 16 × 16 CGRAs) and automatic premapping mechanism (96.6% mapping quality) make it suitable for real-world applications like edge computing and autonomous systems.

Future directions include refining shape-generation algorithms and enhancing the premapper's generality. By bridging the gap between flexibility, efficiency, and applicability, MultiSky paves the way for next-generation CGRA architectures to meet the demands of dynamic, multitask workloads.

REFERENCES

[1] Y. Zhang, A. Rucker, M. Vilim, R. Prabhakar, W. Hwang, and K. Olukotun, "Scalable interconnects for reconfigurable spatial architectures," in *Proc. 46th Int. Symp. Comput. Archit.*, Jun. 2019, pp. 615–628. [Online]. Available: https://dl.acm.org/doi/10.1145/3307650.3322249

[2] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, and T. Nowatzki, "DSAGEN: Synthesizing programmable spatial accelerators," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit.*, Sep. 2020, pp. 268–281. [Online]. Available: https://dl.acm.org/doi/10.1109/ISCA45697.2020.00032

[3] R. Prabhakar et al., "Plasticine: A reconfigurable architecture for parallel paterns," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, Jun. 2017, pp. 389–402. [Online]. Available: https://dl.acm.org/doi/10.1145/3079856.3080256

[4] M. Vilim, A. Rucker, Y. Zhang, S. Liu, and K. Olukotun, "Gorgon: Accelerating machine learning from relational data," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit.*, Sep. 2020, pp. 309–321. [Online]. Available: https://dl.acm.org/doi/10.1109/ISCA45697.2020.00035

[5] A. Rucker, M. Vilim, T. Zhao, Y. Zhang, R. Prabhakar, and K. Olukotun, "Capstan: A vector RDA for sparsity," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchit.*, Oct. 2021, pp. 1022–1035. [Online]. Available: https://dl.acm.org/doi/10.1145/3466752.3480047

[6] O. Hsu et al., "The sparse abstract machine," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, Mar. 2023, pp. 710–726. [Online]. Available: https://dl.acm.org/doi/10.1145/3582016.3582051

[7] A. C. Rucker et al., "Revet: A language and compiler for dataflow threads," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2024, pp. 1–14.

[8] M. Vilim, A. Rucker, and K. Olukotun, "Aurochs: An architecture for dataflow threads," in *Proc. 48th Annu. Int. Symp. Comput. Archit.*, Nov. 2021, pp. 402–415. [Online]. Available: https://dl.acm.org/doi/10.1109/ISCA52012.2021.00039

[9] F. Liu, H. Ahn, S. R. Beard, T. Oh, and D. I. August, "DynaSpAM: Dynamic spatial architecture mapping using out of order instruction schedules," in *Proc. 42nd Annu. Int. Symp. Comput. Archit.*, Jun. 2015, pp. 541–553. [Online]. Available: https://dl.acm.org/doi/10.1145/2749469.2750414

[10] Z. Li, D. Wu, D. Wijerathne, and T. Mitra, "LISA: Graph neural network based portable mapping on spatial accelerators," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Apr. 2022, pp. 444–459.

[11] C. Tan et al., "ARENA: Asynchronous reconfigurable accelerator ring to enable data-centric parallel computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 12, pp. 2880–2892, Dec. 2021.

[12] C. Torng, P. Pan, Y. Ou, C. Tan, and C. Batten, "Ultra-elastic CGRAs for irregular loop Specialization," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2021, pp. 412–425.

[13] L. Liu et al., "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–39, Oct. 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/3357375

[14] "Tx510." Accessed: Feb. 3, 2025. [Online]. Available: http://www.tsingmicro.com/tx510-product.html

[15] S. Kim, Y.-H. Park, J. Kim, M. Kim, W. Lee, and S. Lee, "Flexible video processing platform for 8K UHD TV," in *Proc. IEEE Hot Chips 27 Symp. (HCS)*, Aug. 2015, p. 1. [Online]. Available: https://ieeexplore.ieee.org/document/7477475

[16] "PACT XPP technologies." Accessed: Feb. 3, 2025. [Online]. Available: https://pactxpp.com/

[17] (Intel, Santa Clara, CA, USA). *Tsinghua University and Montage Technology Collaborate to Bring Indigenous Data Center Solutions to China*. Jan. 2016. [Online]. Available: https://www.intc.com/news-events/press-releases/detail/1171/intel-tsinghua-university-and-montage-technology

[18] Y. Ide and N. Yamasaki, "A learning-based fetch thread gating mechanism for a simultaneous multithreading processor," in *Proc. 8th Int. Symp. Comput. Netw. (CANDAR)*, Nov. 2020, pp. 1–10.

[19] S. R. Punyala, T. Marinakis, A. Komaee, and I. Anagnostopoulos, "Throughput optimization and resource allocation on GPUs under multi-application execution," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, Mar. 2018, pp. 73–78.

[20] X. Jin et al., "QoSMT: Supporting precise performance control for simultaneous multithreading architecture," in *Proc. ACM Int. Conf. Supercomput.*, Jun. 2019, pp. 206–216. [Online]. Available: https://dl.acm.org/doi/10.1145/3330345.3330364

[21] C. Zhao, W. Gao, F. Nie, and H. Zhou, "A survey of GPU multitasking methods supported by hardware architecture," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 6, pp. 1451–1463, Jun. 2022.

[22] K. Sankaralingam et al., "TRIPS: A polymorphous architecture for exploiting ILP, TLP, and DLP," *ACM Trans. Archit. Code Optim.*, vol. 1, no. 1, pp. 62–93, Mar. 2004. [Online]. Available: https://dl.acm.org/doi/10.1145/980152.980156

[23] A. Shrivastava, R. Jeyapaul, and A. Shrivastava, "A software scheme for multithreading on CGRAs," *ACM Trans. Embed. Comput. Syst.*, vol. 14, no. 1, pp. 1–26, Jan. 2015. [Online]. Available: https://dl.acm.org/doi/10.1145/2638558

[24] K. Wu, A. Kanstein, J. Madsen, and M. Berekovic, "MT-ADRES: Multithreading on coarse-grained reconfigurable architecture," in *Reconfigurable Computing: Architectures, Tools and Applications*, P. C. Diniz, E. Marques, K. Bertels, M. M. Fernandes, and J. M. P. Cardoso, Eds., Berlin, Germany: Springer, 2007, pp. 26–38, doi: 10.1007/978-3-540-71431-6_3.

[25] T. V. Aa, M. Palkovic, M. Hartmann, P. Raghavan, A. Dejonghe, and L. Van der Perre, "A multi-threaded coarse-grained array processor for wireless baseband," in *Proc. IEEE 9th Symp. Appl. Spec. Process. (SASP)*, Jun. 2011, pp. 102–107.

[26] Q. Zhu, Y. Cao, Y. Qiu, X. Gao, W. Yin, and L. Wang, "A dynamic partial reconfigurable CGRA framework for multi-kernel applications," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, Dec. 2023, pp. 298–299. [Online]. Available: https://ieeexplore.ieee.org/document/10416089

[27] Z. Li, D. Wijerathne, X. Chen, A. Pathania, and T. Mitra, "ChordMap: Automated mapping of streaming applications onto CGRA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 2, pp. 306–319, Feb. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9351547

[28] C. Tan et al., "DRIPS: Dynamic rebalancing of pipelined streaming applications on CGRAs," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Apr. 2022, pp. 304–316.

[29] K. Feng et al., "Amber: A 16-nm system-on-chip with a coarse-grained reconfigurable array for flexible acceleration of dense linear algebra," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 947–959, Mar. 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10258121

[30] T. Kong, K. Koul, P. Raina, M. Horowitz, and C. Torng, "Hardware abstractions and hardware mechanisms to support multi-task execution on coarse-grained reconfigurable arrays," 2023, *arXiv:2301.00861*.

[31] Y. Yang, C. Xie, L. Liu, and X. Peng, "DynMap: A heuristic dynamic mapper for CGRA multi-task dynamic resource allocation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 8, pp. 2979–2991, Aug. 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10869373/?arnumber=10869373

[32] "Clang C language family Frontend for LLVM." Accessed: Feb. 3, 2025. [Online]. Available: https://clang.llvm.org/

[33] "The LLVM compiler infrastructure project." Accessed: Feb. 3, 2025. [Online]. Available: https://llvm.org/

[34] C. Tan, C. Xie, A. Li, K. J. Barker, and A. Tumeo, "OpenCGRA: An open-source unified framework for modeling, testing, and evaluating CGRAs," in *Proc. IEEE 38th Int. Conf. Comput. Design (ICCD)*, Oct. 2020, pp. 381–388.

[35] C. Tan et al., "DynPaC: Coarse-grained, dynamic, and partially reconfigurable array for streaming applications," in *Proc. IEEE 39th Int. Conf. Comput. Design (ICCD)*, Oct. 2021, pp. 33–40.

[36] C. Tan et al., "ICED: An integrated CGRA framework enabling DVFS-aware acceleration," in *Proc. 57th IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Nov. 2024, pp. 1338–1352. [Online]. Available: https://ieeexplore.ieee.org/document/10764442

[37] H. Hu, D. Fang, W. Li, B. Yuan, and J. Hu, "Systolic array placement on FPGAs," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10323742

[38] F. Yang, N. Li, L. Wang, P. Jiang, X. Miao, and X. Wang, "ISARA: An island-style systolic array reconfigurable accelerator based on memristors for deep neural networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 4, pp. 963–975, Apr. 2025. (VLSI) Syst.. [Online]. Available: https://ieeexplore.ieee.org/document/10891373

[39] R. Xu, S. Ma, Y. Guo, and D. Li, "A survey of design and optimization for systolic array-based DNN accelerators," *ACM Comput. Surv.*, vol. 56, no. 1, pp. 1–37, Aug. 2023. [Online]. Available: https://doi.org/10.1145/3604802

[40] Y. Zhang, N. Zhang, T. Zhao, M. Vilim, M. Shahbaz, and K. Olukotun, "SARA: Scaling a reconfigurable dataflow accelerator," in *Proc. 48th Annu. Int. Symp. Comput. Archit.*, Nov. 2021, pp. 1041–1054. [Online]. Available: https://dl.acm.org/doi/10.1109/ISCA52012.2021.00085

[41] X. Man, J. Zhu, G. Song, S. Yin, S. Wei, and L. Liu, "CaSMap: Agile mapper for reconfigurable spatial architectures by automatically clustering intermediate representations and scattering mapping process," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, Jun. 2022, pp. 259–273. [Online]. Available: https://dl.acm.org/doi/10.1145/3470496.3527426

[42] X. Kong et al., "MapZero: Mapping for coarse-grained reconfigurable architectures with reinforcement learning and monte-carlo tree search," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–14. [Online]. Available: https://dl.acm.org/doi/10.1145/3579371.3589081

[43] D. Liu, S. Yin, L. Liu, and S. Wei, "Polyhedral model based mapping optimization of loop nests for CGRAs," in *Proc. 50th Annu. Design Autom. Conf.*, May 2013, pp. 1–8. [Online]. Available: https://dl.acm.org/doi/10.1145/2463209.2488757

[44] J. Gu, S. Yin, and S. Wei, "Stress-aware loops mapping on CGRAs with considering NBTI aging effect," in *Proc. 54th Annu. Design Autom. Conf.*, Jun. 2017, pp. 1–6. [Online]. Available: https://dl.acm.org/doi/10.1145/3061639.3062322

[45] S. Yin, D. Liu, L. Sun, L. Liu, and S. Wei, "DFGNet: Mapping dataflow graph onto CGRA by a deep learning approach," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2017, pp. 1–4. [Online]. Available: https://ieeexplore.ieee.org/document/8050274

[46] L. Chen and T. Mitra, "Graph minor approach for application mapping on CGRAs," in *Proc. Int. Conf. Field-Program. Technol.*, Dec. 2012, pp. 285–292.

[47] D. Wijerathne, Z. Li, A. Pathania, T. Mitra, and L. Thiele, "HiMap: Fast and scalable high-quality mapping on CGRA via hierarchical abstraction," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, Feb. 2021, pp. 1192–1197. [Online]. Available: https://ieeexplore.ieee.org/document/9473916

[48] M. Karunaratne, C. Tan, A. Kulkarni, T. Mitra, and L.-S. Peh, "DNestMap: Mapping deeply-nested loops on ultra-low power CGRAs," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, Jun. 2018, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/document/8465833
[49] T. K. Bandara, D. Wu, R. Juneja, D. Wijerathne, T. Mitra, and L.-S. Peh, "FLEX: Introducing flexible execution on CGRA with spatio-temporal vector dataflow," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9. [Online]. Available: https://ieeexplore.ieee.org/document/10323612
[50] L. Liu, X. Man, J. Zhu, S. Yin, and S. Wei, "Pattern-based dynamic compilation system for CGRAs with online configuration transformation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 12, pp. 2981–2994, Dec. 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9134964
[51] L. Tianyang, Z. Fan, G. Wei, S. Mingqian, and C. Li, "A survey: FPGA-based dynamic scheduling of hardware tasks," *Chin. J. Electron.*, vol. 30, no. 6, pp. 991–1007, 2021. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1049/cje.2021.07.021
[52] M. H. Quraishi, E. B. Tavakoli, and F. Ren, "A survey of system architectures and techniques for FPGA Virtualization," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 9, pp. 2216–2230, Sep. 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9369140/citations?tabFilter=papers#citations
[53] K. Vipin and S. A. Fahmy, "FPGA dynamic and partial reconfiguration: A survey of architectures, methods, and applications," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–39, Jul. 2018. [Online]. Available: https://dl.acm.org/doi/10.1145/3193827
[54] "MEMORY HiCORDER MR6000 Hioki." Accessed: Feb. 3, 2025. [Online]. Available: https://www.hioki.com/global/products/data-acquisition/daq-testing/id_6671
[55] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in *Proc. 54th Annu. Design Autom. Conf.*, Jun. 2017, pp. 1–6. [Online]. Available: https://dl.acm.org/doi/10.1145/3061639.3062262
[56] L. Liu, Z. Li, C. Yang, C. Deng, S. Yin, and S. Wei, "HReA: An energy-efficient embedded dynamically reconfigurable fabric for 13-dwarfs processing," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 65, no. 3, pp. 381–385, Mar. 2018. [Online]. Available: https://ieeexplore.ieee.org/document/7983395
[57] L.-N. Pouchet. "Polybench." Nov. 2023. [Online]. Available: https://www.cs.colostate.edu/ pouchet/software/polybench/
[58] "Kintex 7 FPGAs." [Online]. Available: https://www.amd.com/zh-cn/products/adaptive-socs-and-fpgas/fpga/kintex-7.html
[59] (Intel, Santa Clara, CA, USA). *12th Generation Intel® Core™ i7 Processors*. [Online]. Available: https://www.intel.com/content/www/us/en/ark/products/series/217837/12th-generation-intel-core-i7-processors.html
[60] C. Tan, "StreamingBench," May 2023 [Online]. Available: https://github.com/THUNLP-MT/StreamingBench
[61] C. Xie, X. Li, Y. Hu, H. Peng, M. Taylor, and S. L. Song, "Q-VR: System-level design for future mobile collaborative virtual reality," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2021, pp. 587–599. [Online]. Available: https://doi.org/10.1145/3445814.3446715
[62] C. Tan, D. Patil, A. Tumeo, G. Weisz, S. Reinhardt, and J. Zhang, "VecPAC: A vectorizable and precision-aware CGRA," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2023, pp. 1–9.

Yufei Yang received the M.Sc. degree from the University of Sydney, Camperdown, NSW, Australia, in 2022. He is currently pursuing the Ph.D. degree with the Harbin Institute of Technology, Harbin, China.

His research interests include the architecture and compiler support for CGRA multitask dynamic resource allocation.

Chenhao Xie received the M.S degree in electrical engineering from Washington University in St.louis, St. Louis, MO, MO, USA, in 2013, and the Ph.D. degree from the University of Houston, Houston, TX, USA, in 2018.

He is currently an Associate Professor with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include computer architecture, computer graphics, high performance computing, energy-efficient computing on mobile devices, and memory systems for heterogeneous processor.

Rui Wang (Senior Member, IEEE) received the B.S. and M.S. degrees in computer science from Xi'an Jiaotong University, Xi'an, China, in 2000 and 2003, respectively, and the Ph.D. degree in computer architecture from Beihang University, Beijing, China, in 2009.

He is currently an Associate Professor with the School of Computer Science and Engineering, Beihang University. His current research interests include computer architecture and deep learning compiler.

Liansheng Liu (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees from the Harbin Institute of Technology (HIT), in 2006, 2008, and 2017, respectively.

He is currently an Associate Professor with HIT. From 2012 to 2014, he was with McGill University, Montreal, QC, Canada, as a visiting Ph.D. student supported by the China Scholarship Council. His research aims to develop advanced approaches for industrial application. His research interests include energy-efficient computing, sensor data fusion, and cyber–physical systems.

Xiyuan Peng received the Ph.D. degree in instrumentation science and technology from the Harbin Institute of Technology, Harbin, China, in 1992.

He is currently a Full Professor with the Department of Test and Control Engineering, School of Electronics and Information Engineering, Harbin Institute of Technology. His research interests include automatic test technologies, advanced diagnostics, and prognostics.

Yu Peng received the Ph.D. degree in instrumentation science and technology from the Harbin Institute of Technology (HIT), Harbin, China, in 2004.

He is currently a Full Professor and the Dean of the School of Electronics and Information Engineering, HIT. His current research interests include automatic test technologies, virtual instruments, system health management, and reconfigurable computing.